

Comunicación Entre Procesos

Los procesos ejecutándose concurrentemente en el sistema operativo puede ser procesos independientes o procesos cooperando. Se dice que un proceso es independiente cuando no comparte Datos con cualquier otro proceso en ejecución punto se dice que un proceso es cooperativo cuando éste puede ser afectado o afectar a otros procesos.

Existen varias razones para proporcionar un ambiente que permite la cooperación entre procesos:

Compartir información: dado que varias aplicaciones pudieran estar interesadas en la misma información, por ejemplo, copy y paste, debemos proporcionar un ambiente que permita el acceso concurrente a la información.

Acelerar Computación: si deseamos que una aplicación se ejecute más rápido, debemos hola desde el aire dividirla en subtarefas, las cuales se estarán ejecutando en paralelo con otras. En este caso la o la aceleración sólo se alcanzará si la computadora tiene múltiples núcleos de procesamiento.

Modularidad: el sistema se puede construir de forma modular, dividiendo los Funciones del sistema en procesos separados.

La cooperación entre procesos requiere un proceso de intercomunicación (IPC), el cual es un mecanismo que permite el intercambio de Datos. Existen 2 modelos fundamentales de comunicación entre procesos uno es la memoria compartida y el otro es el pase de mensajes.

Modelo memoria compartida

En este modelo se establece una región de la memoria que será compartida por los procesos, estos pueden intercambiar información leyendo y escribiendo Datos en dicha región

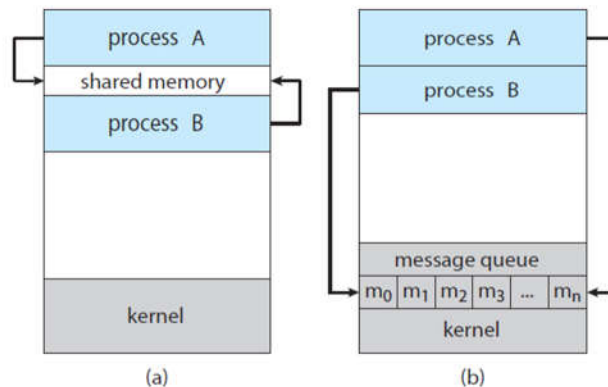


Figure 3.11 Communications models. (a) Shared memory. (b) Message passing.

Comúnmente, una región de memoria compartida reside en el espacio de memoria del proceso que crea el segmento compartido. Otros procesos que deseen comunicarse usando esta memoria compartida deben incluirla en su espacio de memoria. Hay que recordar que el sistema operativo trata de prevenir que un proceso acceda al espacio de memoria de otro proceso por lo que la memoria compartida requiere que los procesos accedan a quitar esta restricción.

Para explicar la memoria compartida recurriremos un problema común llamado productor consumidor. Un productor es un proceso que produce información que es utilizada por un

consumidor. Una solución a este problema es utilizar una cola circular con 2 apuntadores uno al inicio que sería el consumidor y otro al final que sería el productor.

```
#define BUFFER SIZE 10
typedef struct {
    ...
} item;
item buffer[BUFFER SIZE];
int in = 0;
int out = 0;
```

Pipes ordinarios

Un pipe o tubería actúa como un conducto que permite la comunicación entre 2 procesos. Los pipes fueron uno de los primeros mecanismos de comunicación en los sistemas.

El símbolo | que utilizamos para mandar la salida de un proceso a la entrada de otro, es decir, una pipe actúa como un conducto que permite la comunicación entre 2 procesos. Las pipas ordinarias permiten que 2 procesos se comuniquen en una forma estándar productor o consumidor: el productor escribe en un extremo de la pipa y el consumidor lee en el otro extremo. Como resultado las pipas ordinarias son unidireccionales, permiten la comunicación en un solo sentido punto sí se necesita comunicación en ambos sentidos entonces es necesario utilizar 2 pipas cada una enviando información en diferentes sentidos. Típicamente un proceso padre crea el pipe y lo usa para comunicarse con un proceso hijo, que crea a través de un fork.

La función para crear un pipe o entubamiento es pipe();

Prototipo: `int pipe( int fd[2] );`

Regresa: 0 cuando se ejecuta exitosamente.

-1 cuando ocurre un error:

En el prototipo mencionado anteriormente fd es el arreglo de descriptores fd[0] es para lectura y fd[1] es para escritura.

Ejemplo 1 pipe1.c

Crear Pipe

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void) {
    int pipefds[2];

    if(pipe(pipefds) == -1) {
        perror("pipe");
        exit(EXIT_FAILURE);
    }
    printf("Read File Descriptor Value: %d\n", pipefds[0]);
    printf("Write File Descriptor Value: %d\n", pipefds[1]);

    return EXIT_SUCCESS;
}
```

Ejemplo 2 pipe2.c

Lectura escritura en un pipe

```
// Pipe 2
// Escribe una cadena en el descriptor de escritura y la lee en el de lectura

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
int main(void) {
    int pipefds[2];
    char buffer[5];

    if(pipe(pipefds) == -1) {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    char *cad = "hola\0";

    printf("Escribiendo al pipe...\n");
    write(pipefds[1], cad, 5);
    printf("Listo.\n\n");
    printf("Leyendo desde el pipe...\n");
    read(pipefds[0], buffer, 5);
    printf("Hecho.\n\n");
    printf("Palabra leida del pipe: %s\n", buffer);
    return EXIT_SUCCESS;
}
```

Ejemplo 3 Lectura – escritura entre procesos a través de un pipe

//Pipe3.c

//Comunicacion entre padre hijo con un pipe

#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

// 0 - lectura

// 1 - escritura

int main(void) {

int pipefds[2];

char *cad;

char buffer[5];

if(pipe(pipefds) == -1) {

perror("pipe");

exit(EXIT_FAILURE);

}

pid_t pid = fork();

if(pid == 0) { // Proceso Hijo

cad = "FCQI\0";

close(pipefds[0]);

write(pipefds[1], cad, 5);

printf("Enviando escuela del hijo al padre...\n");

sleep(2);

exit(EXIT_SUCCESS);

}

if(pid > 0) { // en proceso padre

wait(NULL);

close(pipefds[1]);

read(pipefds[0], buffer, 5);

close(pipefds[0]);

printf("El padre recibió la cadena '%s'\n", buffer);

}

return EXIT_SUCCESS;

}

Ejemplo 4 Comunicación padre hijo

```
// pipe4 Paso de info capturada del hijo al padre
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

// 0 - lectura
// 1 - escritura

int main(void)
{
    int pipefds[2];
    char *cad;
    char buffer[5];

    if(pipe(pipefds) == -1)
    {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    pid_t pid = fork();

    if(pid == 0) // Proceso Hijo
    { printf("Como te llamas?");
      gets(cad);
      close(pipefds[0]);
      write(pipefds[1], cad, 5);
      printf("Enviando nombre al padre...\n");
      sleep(2);
      exit(EXIT_SUCCESS);
    }

    if(pid > 0) // en proceso padre
    { wait(NULL);
      close(pipefds[1]);
      read(pipefds[0], buffer, 5);
      close(pipefds[0]);
      printf("El hijo se llama '%s'\n", buffer);
    }
}
```

```
    return EXIT_SUCCESS;
}
```

Ejemplo 5 Comunicación padre hijo con datos desde una función

// Pipe 5 el hijo envia al padre un valor obtenido de una funcion

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#define TAM 4

void generar(char []);

int main(void)
{
    char nip[TAM+1];
    char buffer[TAM+1];
    int pipefds[2];
    while(1)
    {
        pipe(pipefds);
        pid_t pid = fork();
        if(pid == 0) //En el hijo
        {
            generar(nip);
            close(pipefds[0]); // cerrar lectura
            write(pipefds[1], nip, TAM + 1);
            generar(nip);
            close(pipefds[0]); // cerrar lectura
            write(pipefds[1], nip, TAM + 1);

            printf("Generando NIP en el hijo \n");

            sleep(2);
            exit(EXIT_SUCCESS);
        }

        if(pid > 0) //En el padre
        {
            wait(NULL);
            close(pipefds[1]); // cierre de escritura
            read(pipefds[0], buffer, TAM+1);
            close(pipefds[0]); // cierre de lectura
            printf("NIP recibido en el padre %s \n" , buffer);
        }
    }
}
```

```

}

return EXIT_SUCCESS;
}

void generar(char pin[])
{
    srand(getpid() + getppid());
    pin[0] = 49 + rand() % 5;
    for(int i = 1; i < TAM; i++)
    {
        pin[i] = 48 + rand() % 5;
    }
    pin[TAM] = '\0';
}

```

Funciones DUP Y DUP2

La función DUP está definida en la biblioteca unistd.h, sirve para duplicar un descriptor de archivo, el número del nuevo descriptor es el primero que se encuentre disponible, por lo que un archivo puede tener varios descriptors que se pueden usar para leer y escribir. El prototipo de dup es:

```
int dup(int descriptor);
```

La llamada dup2 permite generar un duplicado de un descriptor existente, tiene el siguiente prototipo:

```
int dup2(int original, int duplicado);
```

Dup y Dup2 devuelven -1 en caso de error.

Al inicio de un programa, los descriptors asociados con los flujos stdin, stdout y stderr son 0,1 y 2, respectivamente. También están asociados con los símbolos del preprocesador definidos en unistd.h

Descriptor	Flujo	Símbolo
0	stdin	STDIN_FILENO
1	stdout	STDOUT_FILENO
2	stderr	STDERR_FILENO

Ejemplo DUP2

```
// prueba dup y dup 2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <fcntl.h>

int main(void)
{ int fd;

  fd=open("SO2023-2.txt", O_WRONLY | O_CREAT | O_TRUNC, 0600);

  if (fd == -1 )
  {
    perror("No se puede abrir");
    exit(EXIT_FAILURE);
  }
  if (dup2(fd,STDOUT_FILENO) == -1 )
  {
    perror("no se pudo duplicar");
    exit(EXIT_FAILURE);
  }
  if (close(fd) == -1 )
  {
    perror("Error al cerrar");
    exit(EXIT_FAILURE);
  }
  printf("Esta linea se guarda en el archivo SO2023-2.txt");
  return 0;
}
```

Ejemplo 6 Pipe entre dos comandos invocados con execlp

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>
int main(int argc, char* argv [])
{ int fd[2];

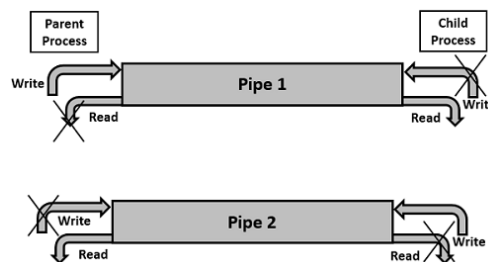
    if (pipe (fd) ==-1)
    { return 1;
    }
    int pid1= fork();
    if (pid1 <0)
    {
        return 2;
    }
    if (pid1 ==0) //proceso hijo
    {
        dup2(fd[1], STDOUT_FILENO);
        close(fd[0]);
        close(fd[1]);
        execlp ("ping","ping", "-c", "5", "148.231.130.237", NULL);
    }

    int pid2= fork();
    if (pid2 <0)
    {
        return 3;
    }
    if (pid2 ==0) //proceso hijo
    {
        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);
        close(fd[1]);
        execlp ("grep","grep", "avg", NULL);
    }
    close(fd[0]);
    close(fd[1]);
    waitpid(pid1,NULL,0);
    waitpid(pid2,NULL,0);
    return 0;
}
```

Full duplex

El establecimiento de un pipe para comunicación en ambos sentidos requiere dos pipes, el algoritmo básico para implementarla es el siguiente:

1. Crear los dos pipes. El primero para que el padre escriba y el hijo lea, el segundo es para que el hijo escriba y el padre lea.
2. Crear proceso hijo
3. Cerrar los extremos que no se van a usar si solo se necesita un extremo.
4. Cerrar los extremos que no se van a usar en el proceso padre, lectura en el pipe1 y escritura en el 2.
5. Cerrar los extremos que no se van a usar en el proceso padre, lectura en el pipe2 y escritura en el 1.
6. Realizar comunicación.



```
//pipe7.c Comunicacion Duplex
#include<stdio.h>
#include<unistd.h>
```

```
int main() {
    int pipefds1[2], pipefds2[2];
    int returnstatus1, returnstatus2;
    int pid;
    char pipe1writemessage[20] = "Hi";
    char pipe2writemessage[20] = "Holis";
    char readmessage[20];
    returnstatus1 = pipe(pipefds1);

    if (returnstatus1 == -1) {
        printf("No se pudo crear pipe 1 \n");
        return 1;
    }
    returnstatus2 = pipe(pipefds2);

    if (returnstatus2 == -1) {
        printf("No se pudo crear pipe 2 \n");
        return 1;
    }
}
```

```
pid = fork();

if (pid != 0) // Padre
{
    close(pipefds1[0]);
    close(pipefds2[1]);
    printf("Padre: Escribiendo a pipe 1: %s\n", pipe1writemessage);
    write(pipefds1[1], pipe1writemessage,
        sizeof(pipe1writemessage));
    read(pipefds2[0], readmessage, sizeof(readmessage));
    printf("Padre: leyendo pipe 2: %s\n", readmessage);
} else { //hijo
    close(pipefds1[1]);
    close(pipefds2[0]);
    read(pipefds1[0], readmessage, sizeof(readmessage));
    printf("Hijo: Leyendo de pipe 1: %s\n", readmessage);
    printf("Hijo: Escribiendo en pipe 2: %s\n", pipe2writemessage);
    write(pipefds2[1], pipe2writemessage,
        sizeof(pipe2writemessage));
}
return 0;
}
```